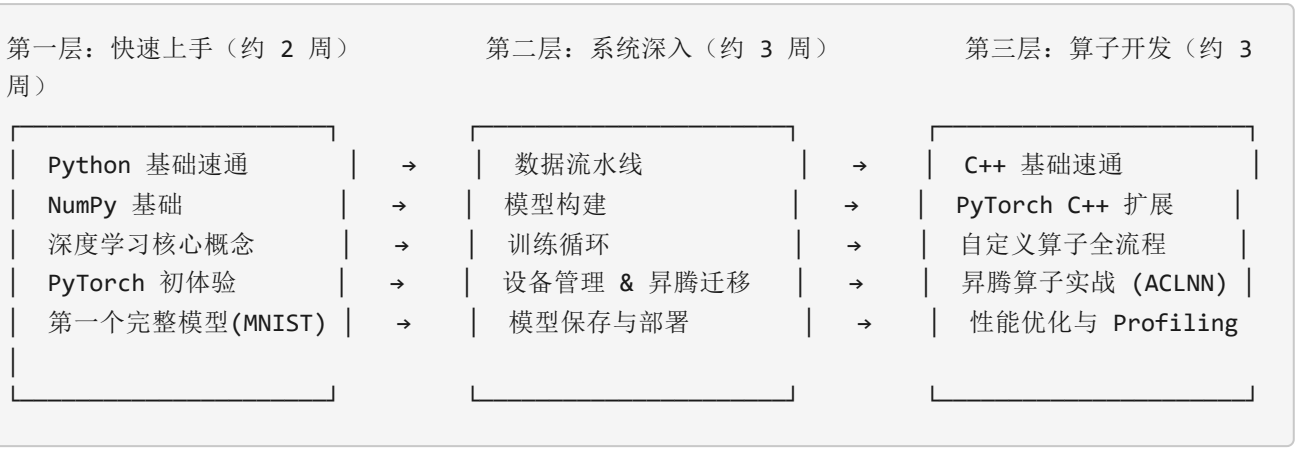
PyTorch 系统学习路线图

> 面向 Ascend NPU 算子开发者的 PyTorch 学习指南
>
> 目标：从 Python 入门到掌握 PyTorch 自定义算子开发，适配昇腾平台 预计周期：**7-8 周**（每天 1-2 小时）

学习路线总览



第一层：快速上手（约 2 周）

> 目标：掌握 Python 基础，理解深度学习核心概念，跑通第一个 PyTorch 模型

1.1 Python 基础速通（4 天）

天数	主题	关键知识点	资源
Day 1	基础语法	数据类型、变量、控制流（`if/for/while`）、函数定义	Python 官方教程 前三章
Day 2	数据结构	`list`、`tuple`、`dict`、`set`，推导式	同上，第五章
Day 3	面向对象	类与对象、继承、`__init__`、特殊方法	第九章
Day 4	实用工具	`pip` 包管理、虚拟环境（`venv/conda`）、文件读写	第七章 + 实践

推荐资源：

- 📖 Python 官方教程 — 精简高效，2-3 天过一遍

- 📖 《Python 编程从入门到实践》（Eric Matthes）— 最推荐的入门书，第 1-10 章
- 🛠 实践：写一个简单的命令行小工具（如计算器、文件分类器）

1.2 NumPy 基础（2 天）

主题	关键知识点
ndarray 创建	`np.array()`、`np.zeros`、`np.ones`、`np.arange`、`np.random`
数组运算	逐元素运算、`np.dot`、`np.matmul`、广播机制
索引与切片	花式索引、布尔掩码、轴操作
统计与变形	`reshape`、`transpose`、`sum/mean/max`、`axis` 参数

推荐资源：

- 📖 NumPy Quickstart Tutorial — 官方快速入门
- 📖 《动手学深度学习》第 2 章（预备知识）— 线性代数部分

1.3 深度学习核心概念（3 天）

天数	主题	核心要点
Day 1	张量与计算图	张量是啥？计算图如何记录运算？前向传播的本质
Day 2	自动微分与梯度下降	链式法则、`backward()`、梯度下降更新参数、学习率
Day 3	损失函数与神经网络	交叉熵损失、MSE、激活函数（ReLU/Sigmoid）、全连接层

推荐资源：

- 🎥 Stanford CS231n — 课程笔记，前几节讲神经网络基础
- 📖 《动手学深度学习》第 3-4 章（线性神经网络 + 多层感知机）
- 🛠 **实践项目：**用 NumPy 手动实现一个线性回归（不用任何框架），理解梯度下降本质

1.4 PyTorch 初体验（3 天）

天数	主题	关键知识点
Day 1	Tensor 基础	`torch.Tensor`、`torch.zeros/ones/rand`、`tensor.shape`、`tensor.dtype`、`tensor.device`
Day 2	Autograd 入门	`requires_grad`、`backward()`、`grad`、`with torch.no_grad()`、`detach()`

天数	主题	关键知识点
Day 3	nn 基础	`nn.Linear`、`nn.ReLU`、`nn.Sequential`、`nn.MSELoss`、`optim.SGD`

PyTorch 与 NumPy 对比：

```
# NumPy
import numpy as np
x = np.array([[1, 2], [3, 4]], dtype=np.float32)
y = x @ x.T  # 矩阵乘法

# PyTorch
import torch
x = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)
y = x @ x.T  # 同样的操作
```

推荐资源：

- 📖 PyTorch 官方 60min 闪电战 — 经典入门必读
- 📖 PyTorch 官方教程 — Tensor 部分

1.5 第一个完整模型（2 天）

项目： 用 PyTorch 实现 MLP 分类器，在 MNIST 手写数字数据集上训练

```python
# 完整训练脚本核心骨架
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# 1. 数据加载
transform = transforms.ToTensor()
train_dataset = datasets.MNIST(root='./data', train=True, transform=transform)
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)

# 2. 模型定义
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Linear(256, 10)
)

# 3. 训练循环
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

for epoch in range(5):
    for images, labels in train_loader:
        outputs = model(images)           # 前向
        loss = criterion(outputs, labels) # 计算损失
        optimizer.zero_grad()             # 梯度清零
        loss.backward()                   # 反向传播
        optimizer.step()                  # 更新参数
    print(f'Epoch {epoch+1}, Loss: {loss.item():.4f}')
```

推荐资源：

- 📖 PyTorch 官方教程 — 图像分类
- 🛠 **Kaggle**：Kaggle Learn 上的 PyTorch 课程

第一层成果检验

- [ ] 能独立写 Python 脚本（类、函数、文件操作、pip 安装包）
- [ ] 理解"张量"概念，能用 `torch.Tensor` 做基本运算
- [ ] 理解自动微分：`requires_grad` + `backward()` 拿到梯度
- [ ] 能跑通 MNIST 完整训练脚本，理解每一行代码的作用

第二层：系统深入（约 3 周）

> 目标：掌握 PyTorch 完整开发流程，能将模型部署到昇腾 NPU 上运行

2.1 数据流水线（3 天）

天数	主题	关键知识点
Day 1	Dataset 与 DataLoader	`torch.utils.data.Dataset`、`DataLoader`、`batch_size`、`shuffle`、`num_workers`
Day 2	自定义 Dataset	继承 `Dataset`、实现 `__len__` 和 `__getitem__`、从文件加载数据
Day 3	数据增强与 Transform	`torchvision.transforms`、`Compose`、`Normalize`、`RandomCrop`、自定义 Transform

推荐资源：

- 📖 PyTorch DataLoader 教程
- 📖 torchvision.transforms 文档

2.2 模型构建（4 天）

天数	主题	关键知识点
Day 1	nn.Module 深入	继承 `nn.Module`、`__init__` 注册子模块、`forward` 方法、`parameters()`
Day 2	常用网络层	`nn.Conv2d`、`nn.BatchNorm2d`、`nn.MaxPool2d`、`nn.Dropout`、`nn.LSTM`
Day 3	经典网络架构	理解 ResNet 残差连接、实现一个简化版 ResNet-18
Day 4	初始化与正则化	权重初始化（`nn.init`）、`weight_decay`、Dropout 原理

推荐资源：

- 📖 PyTorch nn.Module 文档
- 📖 《动手学深度学习》第 7-8 章（卷积神经网络 + 现代网络）
- 🛠 **实践：**用 `nn.Module` 重构之前的 MNIST 代码，使其更模块化

2.3 训练循环（3 天）

天数	主题	关键知识点
Day 1	完整训练框架	训练/验证/测试拆分、`model.train()` vs `model.eval()`、`torch.no_grad()`

天数	主题	关键知识点
Day 2	优化器进阶	`optim.Adam`、`optim.SGD` 动量、学习率调度（`StepLR`、`CosineAnnealingLR`）
Day 3	训练技巧	Early Stopping、Gradient Clipping、Checkpoint 保存最佳模型、TensorBoard 可视化

推荐资源：

- 📖 PyTorch Optimizer 文档
- 📖 PyTorch TensorBoard 教程
- 🛠 **实践项目：**用 CIFAR-10 数据集训练一个分类模型，加入验证集、Checkpoint、学习率衰减

2.4 设备管理与昇腾迁移（3 天）

天数	主题	关键知识点
Day 1	PyTorch 设备抽象	`torch.device`、`.to(device)`、`model.cpu()`/`model.to('cuda')`、`pin_memory`
Day 2	昇腾 NPU 起步	安装 `torch_npu`、`torch.npu.is_available()`、`model.to('npu')`、`tensor.npu()`
Day 3	昇腾混合精度	`torch_npu.npu.amp`、`GradScaler`、FP16 训练注意事项

昇腾迁移示例：

```
import torch
import torch_npu  # 导入 torch_npu 注册 npu 设备

# 检查昇腾 NPU 是否可用
device = torch.device('npu' if torch_npu.npu.is_available() else 'cpu')

# 模型和数据迁移
model = MyModel().to(device)
images, labels = images.to(device), labels.to(device)

# 与 CUDA 代码几乎一致，只需将 cuda 替换为 npu
```

推荐资源：

- 📖 Ascend PyTorch Adapter — 昇腾官方仓库，含安装和示例
- 📖 torch_npu API 文档
- 🛠 **实践：**将上一个 CIFAR-10 模型迁移到 NPU 上运行，对比 CPU vs NPU 速度

2.5 模型保存与部署（2 天）

主题	关键知识点
Checkpoint	`torch.save({'model_state_dict': model.state_dict(), 'optimizer_state_dict': ...}, 'ckpt.pth')`
推理导出	`torch.jit.trace()`、`torch.jit.script()`、ONNX 导出 `torch.onnx.export()`
模型部署	PyTorch 模型在昇腾上的部署：`torch_npu` + ACLNN 推理流程

推荐资源：

- 📖 PyTorch 模型保存与加载
- 📖 PyTorch TorchScript 教程

第二层成果检验

- [ ] 能独立完成一个完整的训练项目（数据加载 → 模型构建 → 训练 → 评估 → 保存）
- [ ] 能在昇腾 NPU 上跑通模型训练
- [ ] 理解 `autograd` 计算图机制
- [ ] 会用 `torch.jit` 导出模型
- [ ] 理解 `model.eval()` + `torch.no_grad()` 的推理规范

第三层：算子开发（约 3 周）

> 目标：掌握 PyTorch 自定义算子开发流程，能独立在昇腾 ACLNN 框架中新增算子

3.1 C++ 基础速通（5 天）

天数	主题	关键知识点
Day 1	基础语法	指针与引用、`new/delete`、`const` 关键字
Day 2	面向对象	类与继承、虚函数、`RAII` 资源管理
Day 3	模板与 STL	函数模板、类模板、`vector`、`string`、`map`
Day 4	CMake 构建	`CMakeLists.txt` 基础、`target_link_libraries`、`find_package`
Day 5	pybind11 入门	绑定 C++ 函数到 Python、`py::class_`、`py::array_t`

推荐资源：

- 🎥 Cherno C++ 系列 — 前 30 集，讲得清楚
- 📖 C++ 教程 | cppreference.com
- 📖 pybind11 文档 — 前几章

3.2 PyTorch C++ 扩展（4 天）

天数	主题	关键知识点
Day 1	环境搭建	`torch.utils.cpp_extension`、`CppExtension`、`CUDAExtension`、`setup.py` 配置
Day 2	编写第一个扩展	C++ 前向函数、`torch::Tensor` 操作、`pybind11` 绑定
Day 3	反向传播	`torch.autograd.Function`、`forward()` + `backward()` 的 C++ 实现
Day 4	调试与测试	编译选项、`torch.autograd.gradcheck` 验证梯度正确性

PyTorch C++ 扩展最小示例：

```
# setup.py
from setuptools import setup
from torch.utils.cpp_extension import CppExtension, BuildExtension

setup(
    name='my_ops',
    ext_modules=[
        CppExtension('my_ops', ['my_ops.cpp']),
    ],
    cmdclass={'build_ext': BuildExtension},
)
```

```
// my_ops.cpp
#include <torch/extension.h>

torch::Tensor my_add(torch::Tensor a, torch::Tensor b) {
    return a + b;
}

PYBIND11_MODULE(my_ops, m) {
    m.def("my_add", &my_add, "Custom add operation");
}
```

推荐资源：

- 📖 PyTorch Custom C++ and CUDA Extensions — 官方教程，必读
- 📖 PyTorch C++ API — `torch::Tensor` 操作文档

3.3 自定义算子全流程（3 天）

天数	主题	关键知识点
Day 1	torch.library 注册	`torch.library.define()`、`torch.library.impl()`、`TORCH_LIBRARY_FRAGMENT`
Day 2	自定义算子模式	Op 定义、Schema 写法、`Tensor` vs `Tensor[]` 类型
Day 3	Autograd 算子	`torch.autograd.Function` 的 Python 实现、`save_for_backward`、梯度公式推导

PyTorch 算子注册示例：

```
import torch
from torch import Tensor

# 方式一：通过 torch.library
torch.library.define("cann_bench::mul", "(Tensor x, Tensor y) -> Tensor")

@torch.library.impl("cann_bench::mul", "cpu")
def mul_impl(x: Tensor, y: Tensor) -> Tensor:
    return x * y

# 方式二：通过 autograd.Function（支持自定义梯度）
class MulFunction(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, y):
        ctx.save_for_backward(x, y)
        return x * y

    @staticmethod
    def backward(ctx, grad_output):
        x, y = ctx.saved_tensors
        return grad_output * y, grad_output * x
```

推荐资源：

- 📖 PyTorch Custom Operators — 官方教程
- 📖 torch.library 文档
- 🛠 **实践**：写一个自定义算子（如 `my_mul`），实现前向和反向，用 `gradcheck` 验证

3.4 昇腾算子实战 — ACLNN 项目（4 天）

结合你的 `aclnn_launch_example` 项目，理解昇腾自定义算子的四层架构。

ACLNN 算子四层架构：

```
┌─────────────────────────────────────────────────┐
│  op_plugin/        Python 绑定层                │
│  mul_plugin.cpp    torch.library + torch_npu 绑定 │
├─────────────────────────────────────────────────┤
│  op_api/           ACLNN API 层                 │
│  aclnn_mul.cpp     L2 API (aclnnMul)            │
│  mul.cpp           L0 API (l0op::Mul)           │
├─────────────────────────────────────────────────┤
│  op_host/          算子宿主定义层               │
│  mul_def.cpp       算子定义 (OP_DEF)            │
│  mul_infershape.cpp Shape推导                   │
│  arch22/mul_tiling.cpp Tiling 策略              │
├─────────────────────────────────────────────────┤
│  op_kernel/        设备侧 Kernel 层             │
│  arch22/mul.h      AscendC kernel 实现          │
│  mul_tiling_data.h  Tiling 数据结构             │
│  mul_tiling_key.h   Tiling Key 定义             │
└─────────────────────────────────────────────────┘
```

学习路径：

天数	主题	内容
Day 1	理解项目结构	通读项目 README、CMakeLists.txt、`func.cmake` 注册机制
Day 2	从 Add 算子入手	精读 `add/` 目录的每个文件，理解算子注册流程
Day 3	对比 Sqrt 算子	看 Sqrt 和 Add 的差异，理解单目 vs 双目算子的不同
Day 4	动手新增算子	按 README 步骤新增 `Mul` 算子，完整走一遍

推荐资源：

- 📖 你的项目 `README.md` — 新增算子步骤（第 59 行起）
- 📖 CANN 开发者文档 — 昇腾社区官方
- 🛠 **实践**：在你的 ACLNN 项目中新增 `Div` 算子，跑通端到端

3.5 性能优化与 Profiling（3 天）

天数	主题	关键知识点
Day 1	Profiling 基础	昇腾 MSVP (MindStudio Vision Profiler)、`torch_npu.profiler`

天数	主题	关键知识点
Day 2	Tiling 策略优化	理解 Tiling 参数对性能的影响、`blockFactor`、`ubFactor` 调优
Day 3	算子融合与内存	融合算子（如 `Add+ReLU` → `AddRelu`）、内存规划、`data_move` 优化

性能分析工具链：

工具	用途	使用场景
`torch_npu.profiler`	PyTorch 层性能分析	模型整体瓶颈定位
MSVP (MindStudio)	昇腾全链路 Profiling	算子级性能分析、Tiling 调优
ACLNN Profiling API	单算子耗时测量	算子微基准测试
`cann_bench` 目录	基准测试框架	你的项目已包含

推荐资源：

- 📖 昇腾社区 性能优化指南
- 📖 AscendC 编程指南 — Tiling 和内存管理部分
- 🛠 **实践**：对同一个算子尝试不同 Tiling 策略，用 Profiler 对比性能差异

第三层成果检验

- [ ] 能用 C++ 写一个 PyTorch 自定义算子并编译通过
- [ ] 理解 `torch.autograd.Function` 的前向/反向实现
- [ ] 理解 ACLNN 的 `op_host → op_kernel → op_api → op_plugin` 四层架构
- [ ] 能独立在 ACLNN 项目中新增一个完整算子
- [ ] 能使用 Profiling 工具定位算子性能瓶颈

推荐学习资源汇总

书籍

书名	作者	推荐理由
《Python 编程从入门到实践》	Eric Matthes	最好的 Python 入门书，看前 10 章
《动手学深度学习》	李沐等	代码和理论并重，有 PyTorch 版，d2l.ai 在线阅读
《深度学习》	Goodfellow 等	“花书”，深度学习理论的圣经，适合作为参考

书名	作者	推荐理由
《PyTorch 深度学习实战》	Harrison Kinsley	代码驱动，适合快速上手

在线课程

课程	平台	说明
CS231n	Stanford	计算机视觉 + 神经网络基础，课程笔记极有价值
CS224n	Stanford	NLP 方向，如果想了解文本处理
PyTorch 官方教程	PyTorch	从入门到高级，覆盖全面
Kaggle Learn	Kaggle	免费 GPU，边学边练
李沐《动手学深度学习》	B 站	配合书籍视频讲解，B 站上搜"李沐 d2l"

官方文档

文档	链接
PyTorch 文档	https://pytorch.org/docs/stable/
torch_npu 文档	https://gitee.com/ascend/pytorch
CANN 开发者文档	https://www.hiascend.com/document
PyTorch C++ API	https://pytorch.org/cppdocs/

实操练习平台

平台	说明	链接
Kaggle	免费 GPU，数据集丰富	kaggle.com
Google Colab	免费 GPU（T4），适合小模型	colab.research.google.com
昇腾社区	昇腾 NPU 云资源	hiascend.com

各阶段实践项目汇总

第一层（快速上手）

项目	难度	说明
用 NumPy 实现线性回归	⭐	手写梯度下降，理解本质
用 PyTorch 实现线性回归	⭐	对比 NumPy 版，感受自动微分
MNIST 手写数字识别	⭐⭐	第一个完整训练脚本，跑通 MLP

第二层（系统深入）

项目	难度	说明
CIFAR-10 图像分类 (ResNet)	⭐⭐⭐	完整训练流程 + 验证 + Checkpoint
迁移到昇腾 NPU	⭐⭐	对比 CPU/NPU 性能差异
模型导出 + 推理部署	⭐⭐	用 TorchScript 导出，写推理脚本

第三层（算子开发）

项目	难度	说明
PyTorch C++ 扩展 (CPU)	⭐⭐⭐	写一个简单算子，编译通过
自定义算子 + 反向传播	⭐⭐⭐⭐	实现 `autograd.Function`，`gradcheck` 验证
ACLNN 新增算子 (Mul/Div)	⭐⭐⭐⭐	在你的项目中完整走一遍新增算子流程
算子性能优化	⭐⭐⭐⭐⭐	Profiling → Tiling 调优 → 性能对比

学习建议

1. **代码 > 理论**：深度学习是实验科学，代码跑通的亲身体会远胜于读十页书。每个阶段至少完成一个完整项目。
2. **先跑通再理解**：不要试图一次性理解所有细节。先跑通 MNIST，再逐步深入 autograd 原理。
3. **善用 AI 工具**：遇到报错或不理解的代码，可以随时问我。但确保自己理解每一行代码的作用，不要无脑复制。
4. **关注昇腾生态**：你的最终目标是昇腾算子开发，学习过程中时刻留意 PyTorch 的哪些 API 对算子开发有直接帮助（`torch.library`、`torch.autograd.Function`、

`torch.utils.cpp_extension`）。

5. **建立调试习惯**：学会用 `print` 调试（`tensor.shape`、`tensor.dtype`）、`torch.autograd.set_detect_anomaly(True)` 排查梯度问题。
6. **阅读官方文档**：PyTorch 官方文档质量很高，遇到问题优先查文档，而不是搜索引擎。

Happy Hacking! 🚀